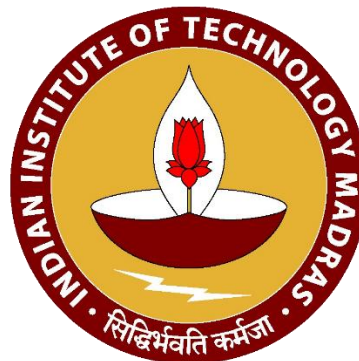


Python for Data Science

Web M. Tech Course DA5301W

Module 2: Python Environments and PEP Standards

Dr. P.S Jayadev



Contents

- Versions in Python
- Modules, Packages and Libraries in Python
- Dependencies and Requirements
- Python Virtual Environments
- Tools for Managing Virtual Environments
- Python Scripts Vs Python Notebooks
- PEP Standards and Linting

Motivation for this Module

- **Reproducibility** : Same code + same data = same results
- **Experimentation** : ML requires quick library/model switching
- **Collaboration** : Ensuring all teams, testing and deployment environments are consistent
- **Portability** : From Jupyter prototyping to production APIs, Docker, cloud deployment
- This module is one of the foundation aspects of professional AI/DS practice

Versions in Python

- Python Software Foundation (PSF) oversees Python's development and releases.
- Development work includes:
 - Adding new features (syntax, libraries, performance improvements)
 - Fixing bugs and security issues
 - Deprecating outdated features
 - Releasing updates regularly based on community proposals
- Python follows semantic versioning:
 - Major version → Python 3
 - Minor version → Python 3.11, 3.12
 - Micro/patch version → Python 3.12.2 (bug fixes, security updates)
- 3.13.3 is the latest stable version of python released in April 2025

Modules, Packages and Libraries

Modules, Packages and Libraries in Python

- **Python Module:**

- A single Python file (.py) containing code, such as functions, classes, or variables
- Modules help you organize and reuse code by allowing import of the file in other programs

- **Python Package:**

- A folder containing multiple modules a special `__init__.py` file
- Packages help group related modules together to keep large projects organized

Modules, Packages and Libraries in Python

- **Python Sub-Package:**

- A folder within a package that contains its own `__init__.py` and modules or further sub-packages
- Sub-packages create hierarchical organization of modules within packages

- **Python Sub-Module:**

- A module placed inside a sub-package
- Sub-modules are Python files (.py) nested deeper inside the package hierarchy

Modules, Packages and Libraries in Python

- **Python Library:**

- A collection of modules and/or packages offering various functionalities as a whole
- Libraries include popular packages like NumPy, pandas, which provide wide-ranging tools

```
numpy/  
├── __init__.py  
├── _core/  
│   ├── __init__.py  
│   ├── multiarray.py  
│   ├── umath.py  
│   └── ...  
├── lib/  
│   ├── __init__.py  
│   ├── type_check.py  
│   ├── scimath.py  
│   └── ...  
├── linalg/  
│   ├── __init__.py  
│   ├── linalg.py  
│   └── ...  
├── fft/  
│   ├── __init__.py  
│   ├── fft.py  
│   └── ...  
├── random/  
│   ├── __init__.py  
│   ├── _generator.py  
│   └── ...
```

Example of Numpy Library in Python

- **Package:** `numpy` - the top-level folder containing many modules and sub-packages.
- **Sub-Package:** `numpy.linalg` - folder inside `numpy` organizing linear algebra-related code
- **Sub-Module:** `numpy.linalg.linalg` - Python file module in the `numpy.linalg` sub-package.
- **Module (not a sub-module):** `numpy.fft` - Module directly inside the `numpy` package for Fourier transforms
- Reference on how to create a package in python: <https://py-pkgs.org/04-package-structure.html>

Example of Numpy Library in Python

- **Q:** What is contained in numpy library apart from numpy package?
 - Compiled binary extensions (C/C++/Fortran) for fast numerical computation.
 - Additional resources, metadata, and scripts supporting building, testing, and package management
 - Dependencies and supporting files that ensure full functionality and performance

Versions in Python Libraries

- Python Libraries also have versions just like python itself
 - Eg: NumPy 1.23 vs NumPy 1.26
- Further, Library compatibility is tied to Python version
 - **Example:** TensorFlow 2.11 works only up to Python 3.10
- Therefore, when choosing Python version, you must also check library versions
- Mismatch results in errors, installation failures and inconsistent results

Dependencies and Requirements

What are Dependencies?

- **Dependency:** Any python packages or library your project relies on
- **Example:**
 - NumPy : for arrays related operations
 - pandas : for data wrangling
 - scikit-learn : for ML models
- AI/DS projects may have dozens of dependencies
- Dependencies also have their own dependencies leading to a dependency tree
- **Example:** scikit-learn depends on numpy, scipy, and joblib libraries

What are Requirements?

- List of specific dependencies (libraries and their versions) needed for your project
- Generally managed as requirements.txt file
- Eg:

```
numpy==1.26.2  
pandas==2.0.3  
scikit-learn==1.3.0
```
- Requirements file is very critical for reproducible results
- Without requirements: one teammate may run on numpy 1.25, another on numpy 1.26 – may lead to inconsistent outputs

Problem of Dependency Conflicts

- Different projects often need different versions of the same library
 - Each project may rely on a feature, bug fix, or behavior specific to a certain library version
 - Newer versions may remove deprecated functions, change APIs, or optimize performance
- **Example:**
 - Project A might need `torch==2.0`
 - Project B might need `torch==1.12`
- **Issue:** Installing two versions of same library can results in conflicts & broken projects
- **Solution:** Isolated Environments is the way to go

Python Virtual Environments

Python Virtual Environments

- **Virtual Environment:** Isolated space in a system with its own Python version and dependencies installed
- **Benefits:**
 - Each project can have its virtual environment with its own library versions
 - No clashes between projects
 - Easy to replicate setup on another machine or server
- **Tools to manage Python environments:**
 - **venv:** Simple built-in Python tool for creating Python environments
 - **poetry:** Python dependency and project manager that simplifies packaging, publishing, and environment handling for projects.
 - **conda:** Powerful environment and package manager that handles both Python and non-Python dependencies (Say CUDA)
 - **uv:** Fast, modern alternative offering improved speed and streamlined environment management.

Python Virtual Environments

Feature/Tool	venv	conda	uv	poetry
Scope	Python-only environments	Python + non-Python dependencies (CUDA,etc.)	Python-only environments	Python-only dependencies + package management
Speed	Moderate	Slower (complex solver)	Very fast (Rust-based)	Moderate
Ease of Use	Simple, good for beginners	Rich features but heavier	Easy with modern CLI	Structured workflow, but has learning curve
Reproducibility	Basic: Ensures only dependency list is captured (requirements.txt). May not guarantee reproducibility (across OS, libraries with C,C++ dependencies)	Strong: Captures versions and environment details more fully (environment.yml)	Basic (requirements.txt)	Strong: Provides deterministic builds by pinning exact versions, ensuring reproducibility across systems (pyproject.toml)
AI/DS Use Case	For small Python-only projects	ML/AI with heavy non-python dependencies (CUDA)	Fast set up and Python-only DS workflows	Ideal for Python data science projects, package management, and publishing to PyPI

Lifecycle of Using Environments

Create → Set up a new environment for your project

Activate → Switch into that environment to work

Install → Add dependencies/libraries you need

Use → Run scripts, notebooks, or ML models inside the environment

Share → Export requirements (requirements.txt) so others can replicate

Reuse/Move → Rebuild the same environment on another machine or cloud

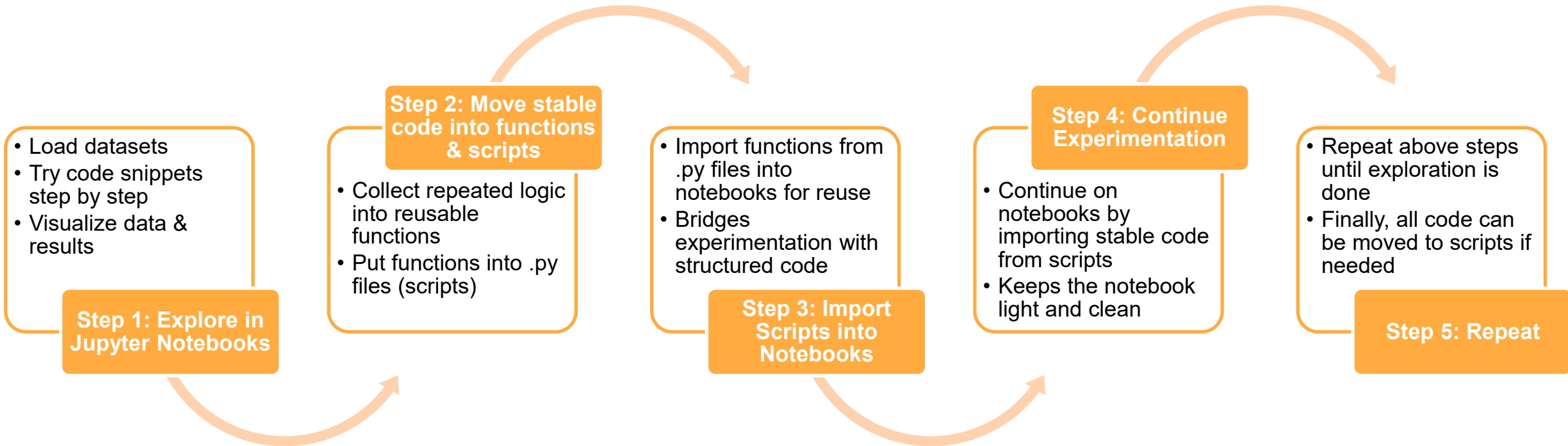
Switch/Deactivate → Move between environments depending on project needs

Jupyter Notebooks Vs Python Scripts

Python Notebooks Vs Python Scripts

Aspect	Python Notebooks	Python Scripts
Format	.ipynb (interactive, cell-based) Jupyter notebooks or Google Colab	.py (linear code file)
Best For	Exploration, EDA, teaching, quick demos	Production code, automation, deployment
Interactivity	Highly interactive (cell execution, visual output)	Minimal interactivity (runs top to bottom)
Documentation	Code + Markdown + plots in one place	External docs required
Version Control	Harder to manage (not Git friendly)	Easy (plain text, Git-friendly)
Debugging/Testing	Suitable for manual debugging but limited tools	Manual testing is cumbersome but comprehensive tools are available
AI/DS Use Case	Prototypes, Experimentation, etc.	ML pipelines, APIs, CI/CD, scaling, etc.

From Notebooks to Scripts



PEP Standards and Linting

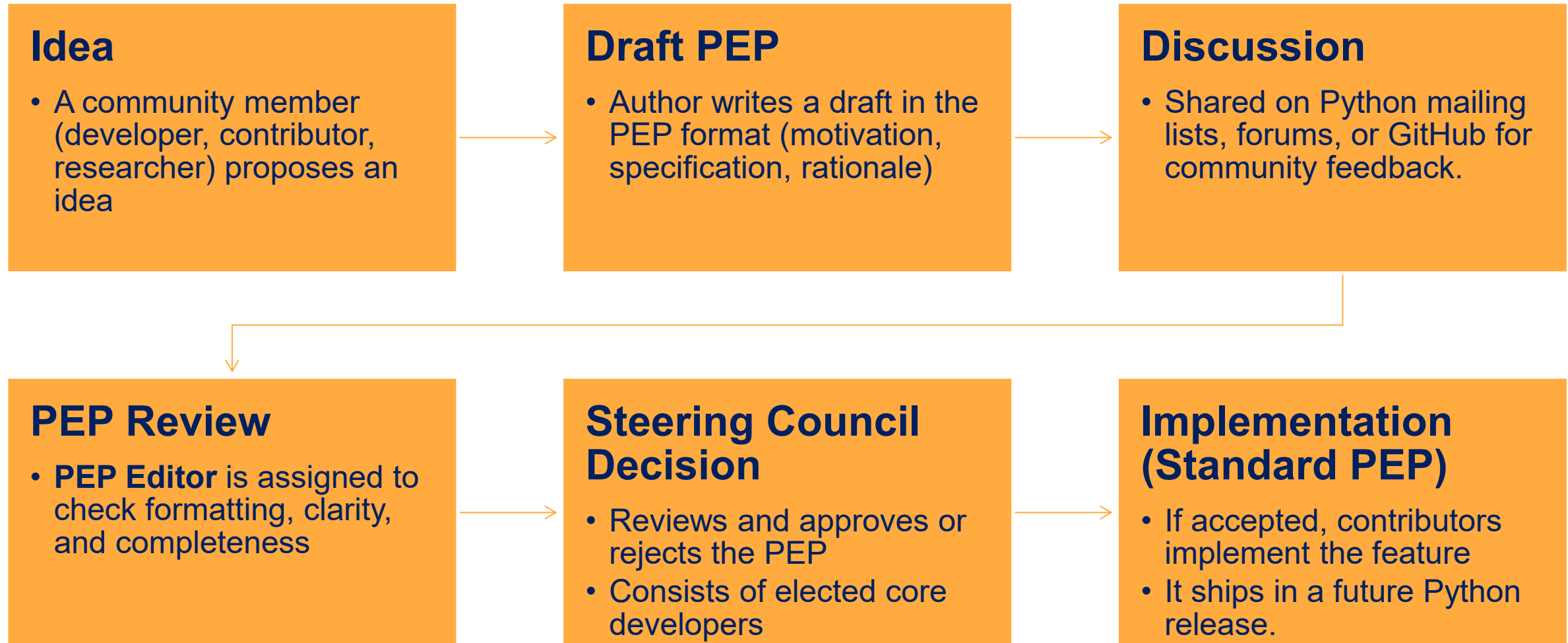
What is PEP?

- PEP = Python Enhancement Proposal
- A design document that describes:
 - New features or changes in Python
 - Guidelines and best practices
 - Python development processes
- Reviewed and approved by the Python community & PSF
- Ensures Python evolves in a structured and transparent way
- Ref: <https://peps.python.org/>

Types of PEP Documents

- **Standards Track PEPs**
 - Propose new features or changes to Python's syntax, libraries, or interpreter
 - Example: PEP 572 – Proposal to introduce assignment expressions (`:=` operator)
- **Informational PEPs**
 - Provide guidelines, conventions, or general information
 - Example: PEP 8 – Style Guide for Python Code (<https://pep8.org/>)
- **Process PEPs**
 - Define or change processes for Python development itself
 - Example: PEP 1 – How PEPs are structured and submitted

PEP Workflow



PEP 8 Style Guide for DS

- **To Ensure Consistency & Readability:** Critical when sharing notebooks, scripts, and ML pipelines
- **Key points:**
 - **Naming conventions:** snake_case for variables/functions, CamelCase for classes
 - **Indentation:** 4 spaces
 - **Line length:** ≤ 79 chars (keeps notebooks & Git differences clean)
 - **Imports:** one per line, grouped logically (standard $\rightarrow$ third-party $\rightarrow$ local)
 - **Whitespace:** use around operators for clarity ($a = b + c$)
 - **Comments & Docstrings:** Explain data transformations, models, assumptions
- Overall improves collaboration, reduces errors, and eases debugging ML pipelines
- IDEs like VSCode aid in following PEP 8 standards

Linting

- **Linting** : Process of checking the code
- **Finds:**
 - Syntax errors
 - Formatting issues
 - Unused variables/imports
 - Style violations based on PEP8 standards
- Helps you write clean, error-free, and consistent code
- In AI&DS projects, linting ensures:
 - Code is readable & sharable across the team
 - Catch bugs early (typos, missing imports, unused code)
 - Smooth transition from exploration → production pipelines

Ruff for Data Science Projects

- What is Ruff?
 - A modern, ultra-fast linter and formatter for Python
 - Written in Rust → blazing fast compared to older tools
 - Combines multiple tools in one
- Features of Ruff relevant to Data Science
 - Unused import/variable detection (can auto-fix)
 - Automatic import sorting: standard → third-party → project-specific imports
 - PEP 8 style enforcement (can auto-fix)
 - Docstring and comment checks
 - Duplicate code detection
 - Nudges to refactor into reusable functions
 - Easy to integrate in VSCode, Jupyter notebook, or CI/CD pipelines

PEP 20 : Zen of Python for DS

- **19 Guiding Principles** for Writing Pythonic Code written by Tim Peters in 1999
- Shapes the philosophy of Python development: simplicity, readability, and clarity
- **“Beautiful is better than ugly”** : Write clean notebooks, clear plots, and well-structured ML code
- **“Simple is better than complex”** : Don’t over-engineer pipelines; keep preprocessing & models understandable
- **“Readability counts”** : Document feature engineering steps; teammates must understand your code
- **“Errors should never pass silently”** : Handle missing data, NaNs, data inconsistencies and model errors explicitly
- **“There should be one—preferably only one—obvious way to do it”** : Follow consistent or standard ML workflows

Summary

- Versions of python and versions of libraries in python are closely connected and compatibility between needs to be ensured for any python project
- Virtual environments are the key to reproducibility, portability and collaborative coding in data science projects
- Different tools can be used to manage virtual environments depending on the needs of the project
- A balanced approach between Python Notebooks and Python scripts can be taken to explore, experiment and manage code properly in data science & AI projects
- PEP documents ensure that python enhancements and standards are managed in a structured manner
- PEP 8 is the global standard for style and conventions to be followed in python coding
- Linting is an automated process to identify potential issues and deviations from coding standards
- Ruff is a popular modern tool to manage and auto-correct issues identified during linting